

JAVA PRACTICAL RECORD

Inheritance and Interfaces — 8 Separate Programs

Experiment 1 — Class Extends Class and Implements Interface

1. Aim / Objective

Demonstrates a class inheriting from another class and implementing an interface.

2. System Requirements

JDK 8 or above, Java compiler, and any Java IDE or command-line terminal.

3. Algorithm / Logic

1. Define the required class/interface structure. 2. Declare variables and methods. 3. Implement or inherit the required methods. 4. Create the object in main(). 5. Call the methods and display the result.

4. UML Class / Interface Diagram

I1 (interface) → C1; C2 → C1

5. Source Code

```
interface I1 {
    int m = 20;

    void display();
}

class C2 {
    int n = 10;
}

class C1 extends C2 implements I1 {

    @Override
    public void display() {
        System.out.println("This is from interface I1");
        System.out.println("Sum of n and m is : " + (n + m));
    }
}

public class C1ExtC2ImpI1 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.display();
    }
}
```

6. Compilation & Execution

```
javac C1ExtC2ImpI1.java
java C1ExtC2ImpI1
```

7. Output Screen

```
This is from interface I1
Sum of n and m is : 30
```

8. Result

The program was executed successfully and the expected output was obtained.

Experiment 2 — Single Inheritance

1. Aim / Objective

Demonstrates single inheritance where class B inherits members of class A.

2. System Requirements

JDK 8 or above, Java compiler, and any Java IDE or command-line terminal.

3. Algorithm / Logic

1. Define the required class/interface structure. 2. Declare variables and methods. 3. Implement or inherit the required methods. 4. Create the object in main(). 5. Call the methods and display the result.

4. UML Class / Interface Diagram

A (parent) → B (child)

5. Source Code

```
class A{
    int x=100;
    void sample1()
    {
        System.out.println("This is from class A");
        System.out.println("The value of x is: "+x);
    }
}
class B extends A{
    int y=200;
    void sample2()
    {
        System.out.println("This is from class B");
        System.out.println("The sum is : "+(x+y));
    }
}
public class C1ExtendsC2 {

    public static void main(String[] args) {
        B obj=new B();
        obj.sample1();
        obj.sample2();
    }
}
```

6. Compilation & Execution

```
javac C1ExtendsC2.java
java C1ExtendsC2
```

7. Output Screen

```
This is from class A
The value of x is: 100
This is from class B
The sum is : 300
```

8. Result

The program was executed successfully and the expected output was obtained.

Experiment 3 — Class Implements Interface

1. Aim / Objective

Demonstrates implementation of an interface by a class.

2. System Requirements

JDK 8 or above, Java compiler, and any Java IDE or command-line terminal.

3. Algorithm / Logic

1. Define the required class/interface structure. 2. Declare variables and methods. 3. Implement or inherit the required methods. 4. Create the object in main(). 5. Call the methods and display the result.

4. UML Class / Interface Diagram

I1 (interface) → C (implements)

5. Source Code

```
interface I1 {
    int n = 10;

    void display();
}

class C implements I1 {

    @Override
    public void display() {
        System.out.println("This is from interface I1");
        System.out.println("Square of n is : " + (n * n));
    }
}

public class C1ImplementsI1 {
    public static void main(String[] args) {

        C obj=new C();
        obj.display();
    }
}
```

6. Compilation & Execution

```
javac C1ImplementsI1.java
java C1ImplementsI1
```

7. Output Screen

```
This is from interface I1
Square of n is : 100
```

8. Result

The program was executed successfully and the expected output was obtained.

Experiment 4 — Multiple Interfaces

1. Aim / Objective

Demonstrates a class implementing two interfaces.

2. System Requirements

JDK 8 or above, Java compiler, and any Java IDE or command-line terminal.

3. Algorithm / Logic

1. Define the required class/interface structure. 2. Declare variables and methods. 3. Implement or inherit the required methods. 4. Create the object in main(). 5. Call the methods and display the result.

4. UML Class / Interface Diagram

I1 + I2 → C1 (implements both)

5. Source Code

```
interface I1{
    int x=10;
    void sample1();
}
interface I2
{
    int y=20;
    void sample2();
}
class C1 implements I1,I2{

    @Override
    public void sample1()
    {
        System.out.println("This is from interface I1");
        System.out.println("Square of x is : " + (x*x));
    }
    @Override
    public void sample2()
    {
        System.out.println("This is from interface I2");
        System.out.println("Sum of x and y is : " + (x+y));
    }
}
public class C1ImplementsI1I2 {
    public static void main(String[] args) {
        C1 obj=new C1();
        obj.sample1();
        obj.sample2();
    }
}
```

6. Compilation & Execution

```
javac C1ImplementsI1I2.java
java C1ImplementsI1I2
```

7. Output Screen

```
This is from interface I1
Square of x is : 100
This is from interface I2
Sum of x and y is : 30
```

8. Result

The program was executed successfully and the expected output was obtained.

Experiment 5 — Interface Extends Interface

1. Aim / Objective

Demonstrates interface inheritance, where I1 extends I2.

2. System Requirements

JDK 8 or above, Java compiler, and any Java IDE or command-line terminal.

3. Algorithm / Logic

1. Define the required class/interface structure. 2. Declare variables and methods. 3. Implement or inherit the required methods. 4. Create the object in main(). 5. Call the methods and display the result.

4. UML Class / Interface Diagram

I2 → I1 → C1

5. Source Code

```
interface I2 {
    int n = 10;

    void display();
}

interface I1 extends I2 {
    int m = 20;

    void example();
}

class C1 implements I1 {

    @Override
    public void display() {
        System.out.println("This is from interface I2");
        System.out.println("Value of n is : " + n);
    }

    @Override
    public void example() {
        System.out.println("This is from interface I1");
        System.out.println("Sum of n and m is : " + (n + m));
    }
}

public class I1ExtendsI2 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.display();
        obj.example();
    }
}
```

6. Compilation & Execution

```
javac I1ExtendsI2.java
java I1ExtendsI2
```

7. Output Screen

```
This is from interface I2
Value of n is : 10
This is from interface I1
Sum of n and m is : 30
```

8. Result

The program was executed successfully and the expected output was obtained.

Experiment 6 — Interface Extends Multiple Interfaces

1. Aim / Objective

Demonstrates an interface extending two interfaces and a class implementing the resulting interface.

2. System Requirements

JDK 8 or above, Java compiler, and any Java IDE or command-line terminal.

3. Algorithm / Logic

1. Define the required class/interface structure. 2. Declare variables and methods. 3. Implement or inherit the required methods. 4. Create the object in main(). 5. Call the methods and display the result.

4. UML Class / Interface Diagram

I2 + I3 → I1 → C1

5. Source Code

```
interface I2 {
    int n = 10;

    void display();
}

interface I3 {
    int m = 20;

    void example();
}

interface I1 extends I2, I3 {
    int p = 30;

    void sample();
}

class C1 implements I1 {

    @Override
    public void display() {
        System.out.println("This is from interface I2");
        System.out.println("Value of n is : " + n);
    }

    @Override
    public void example() {
        System.out.println("This is from interface I3");
        System.out.println("Value of m is : " + m);
    }

    @Override
    public void sample() {
        System.out.println("This is from interface I1");
        System.out.println("Sum of n, m and p is : " + (n + m + p));
    }
}

public class I1ExtendsI2I3 {
    public static void main(String[] args) {
        C1 obj = new C1();
        obj.display();
        obj.example();
        obj.sample();
    }
}
```

6. Compilation & Execution

```
javac I1ExtendsI2I3.java
java I1ExtendsI2I3
```

7. Output Screen

```
This is from interface I2
Value of n is : 10
This is from interface I3
Value of m is : 20
This is from interface I1
Sum of n, m and p is : 60
```

Sum of n, m and p is : 60

8. Result

The program was executed successfully and the expected output was obtained.

Experiment 7 — Interface Example 1

1. Aim / Objective

Demonstrates basic interface implementation with arithmetic operations.

2. System Requirements

JDK 8 or above, Java compiler, and any Java IDE or command-line terminal.

3. Algorithm / Logic

1. Define the required class/interface structure. 2. Declare variables and methods. 3. Implement or inherit the required methods. 4. Create the object in main(). 5. Call the methods and display the result.

4. UML Class / Interface Diagram

A (interface) → D

5. Source Code

```
interface A {
    int x=100;
    void display();
}

class D implements A {

    @Override
    public void display()
    {
        int y=200;
        System.out.println("The sum is : "+(x+y));
        System.out.println("The DIfference is : "+(x-y));
    }
}

public class Interfaces1 {
    public static void main(String[] args) {
        D i=new D();
        i.display();
    }
}
```

6. Compilation & Execution

```
javac Interfaces1.java
java Interfaces1
```

7. Output Screen

```
The sum is : 300
The DIfference is : -100
```

8. Result

The program was executed successfully and the expected output was obtained.

Experiment 8 — Multiple Interfaces Example

1. Aim / Objective

Demonstrates a class implementing three interfaces.

2. System Requirements

JDK 8 or above, Java compiler, and any Java IDE or command-line terminal.

3. Algorithm / Logic

1. Define the required class/interface structure. 2. Declare variables and methods. 3. Implement or inherit the required methods. 4. Create the object in main(). 5. Call the methods and display the result.

4. UML Class / Interface Diagram

A + B + C → Demo

5. Source Code

```
interface A
{
    int x=20;
    void sample();
}
interface B
{
    int y=30;
    void example();
}
interface C
{
    int z=40;
    void display();
}
class Demo implements A,B,C
{
    @Override
    public void sample()
    {
        System.out.println("The sum of x and y is : "+(x+y));
        System.out.println("The product of x and z is : "+(x*z));
    }
    @Override
    public void example()
    {
        System.out.println("The difference of z and y is : "+(z-y));
    }
    @Override
    public void display()
    {
        System.out.println("The product of z and y is : "+(z*y));
    }
}
public class Interfaces2 {
    public static void main(String[] args) {

        Demo d=new Demo();
        d.sample();
        d.example();
        d.display();

    }
}
```

6. Compilation & Execution

```
javac Interfaces2.java
java Interfaces2
```

7. Output Screen

```
The sum of x and y is : 50
The product of x and z is : 800
The difference of z and y is : 10
The product of z and y is : 1200
```

8. Result

The program was executed successfully and the expected output was obtained.